Del Álgebra Relacional a SQL

IIC2413 — Bases de Datos

Semana 2 · 2026-1

El **álgebra relacional** define *qué* operaciones podemos hacer sobre relaciones. **SQL** es el lenguaje que implementa esas operaciones en un motor de bases de datos.

Álgebra Relacional	SQL
σ (selección)	`WHERE`
π (proyección)	`SELECT`
× (producto cartesiano)	`FROM T1, T2`
⋈ (join natural)	`NATURAL JOIN`
∪ (unión)	`UNION`
∩ (intersección)	`INTERSECT`
− (diferencia)	`EXCEPT`
ρ (renombramiento)	`AS`

Esquema de ejemplo

Usaremos tres relaciones sobre películas:

```
Peliculas(id, nombre, anio, categoria, calificacion)
Actores(id, nombre, edad)
Actuo_en(id_actor, id_pelicula)
```

Cargadas en una base SQLite: `peliculas.db`

1. Selección — σ

Álgebra relacional

$$\sigma_{\text{categoria='Ciencia Ficción'}}(\text{Peliculas})$$

> Filtra las tuplas que cumplen una condición.

SQL

```sql
SELECT *
FROM Peliculas
WHERE categoria = 'Ciencia Ficción';
```

σ ↔️ WHERE

2. Proyección — π

Álgebra relacional

$$\pi_{\text{nombre, anio}}(\text{Peliculas})$$

> Selecciona columnas (y elimina duplicados).

SQL

```
SELECT DISTINCT nombre, anio
FROM Peliculas;
```

π ↔️ SELECT DISTINCT

> Nota: SQL no elimina duplicados por defecto. Para replicar π, usamos `DISTINCT`.

3. Composición: σ + π

Álgebra relacional

$$\pi_{\text{nombre}}(\sigma_{\text{calificacion}>8.0}(\text{Peliculas}))$$

> Nombres de películas con calificación mayor a 8.

SQL

```sql
SELECT DISTINCT nombre
FROM Peliculas
WHERE calificacion > 8.0;
```

Las operaciones del álgebra se componen; en SQL se combinan cláusulas.

4. Producto Cartesiano — ×

Álgebra relacional

$$\text{Actores} \times \text{Actuo_en}$$

> Todas las combinaciones posibles de tuplas.

SQL

```
SELECT *
FROM Actores, Actuo_en;
```

Resultado: si Actores tiene n filas y Actuo_en tiene m, obtenemos $n \times m$ filas.

5. Join — ⋈

Álgebra relacional

$$\text{Actores} \bowtie_{\text{Actores.id}=\text{Actuo_en.id_actor}} \text{Actuo_en}$$

> Producto cartesiano + selección sobre la condición de join.

SQL

```sql
SELECT *
FROM Actores
JOIN Actuo_en ON Actores.id = Actuo_en.id_actor;
```

⋈ = × + σ, pero el motor lo optimiza.

6. Consulta compuesta: ¿En qué películas actuó cada actor?

Álgebra relacional

$$\pi_{\text{A.nombre, P.nombre}}(\text{Actores} \bowtie_{\text{A.id=AE.id_actor}} \text{Actuo_en} \bowtie_{\text{AE.id_pelicula=P.id}} \text{Peliculas})$$

SQL

```sql
SELECT A.nombre AS actor, P.nombre AS pelicula
FROM Actores A
JOIN Actuo_en AE ON A.id = AE.id_actor
JOIN Peliculas P  ON AE.id_pelicula = P.id;
```

Notar el uso de **alias** (`A` , `AE` , `P`) → renombramiento (ρ).

7. Unión — ∪

Álgebra relacional

$$\pi_{\text{nombre}}(\sigma_{\text{categoria='Drama'}}(\text{Peliculas})) \cup \pi_{\text{nombre}}(\sigma_{\text{categoria='Ciencia Ficción'}}(\text{Peliculas}))$$

SQL

```sql
SELECT nombre FROM Peliculas WHERE categoria = 'Drama'
UNION
SELECT nombre FROM Peliculas WHERE categoria = 'Ciencia Ficción';
```

`UNION` elimina duplicados (como en el álgebra). Para mantenerlos: `UNION ALL`.

8. Intersección — ∩

Álgebra relacional

$$\pi_{\text{id_actor}}(\sigma_{\text{id_pelicula}=1}(\text{Actuo_en})) \cap \pi_{\text{id_actor}}(\sigma_{\text{id_pelicula}=5}(\text{Actuo_en}))$$

> Actores que actuaron en la película 1 **y** en la película 5.

SQL

```sql
SELECT id_actor FROM Actuo_en WHERE id_pelicula = 1
INTERSECT
SELECT id_actor FROM Actuo_en WHERE id_pelicula = 5;
```

9. Diferencia — −

Álgebra relacional

$$\pi_{\text{id_actor}}(\sigma_{\text{id_pelicula}=1}(\text{Actuo_en})) \;-\; \pi_{\text{id_actor}}(\sigma_{\text{id_pelicula}=5}(\text{Actuo_en}))$$

> Actores que actuaron en la película 1 **pero no** en la 5.

SQL

```sql
SELECT id_actor FROM Actuo_en WHERE id_pelicula = 1
EXCEPT
SELECT id_actor FROM Actuo_en WHERE id_pelicula = 5;
```

10. Renombramiento — ρ

Álgebra relacional

$$\rho_{\text{P1}}(\text{Peliculas}) \bowtie_{\text{P1.categoria}=\text{P2.categoria} \,\wedge\, \text{P1.id}\neq\text{P2.id}} \rho_{\text{P2}}(\text{Peliculas})$$

> Pares de películas de la misma categoría.

SQL

```sql
SELECT P1.nombre, P2.nombre, P1.categoria
FROM Peliculas P1, Peliculas P2
WHERE P1.categoria = P2.categoria
  AND P1.id < P2.id;
```

ρ ↔️ **AS** (alias de tabla)

Resumen de equivalencias

Operación	Símbolo	SQL	Cláusula clave
Selección	σ	`SELECT * FROM R WHERE ...`	`WHERE`
Proyección	π	`SELECT DISTINCT cols FROM R`	`SELECT DISTINCT`
Producto cart.	$\times$	`FROM R1, R2`	`,` en `FROM`
Join	$\bowtie$	`R1 JOIN R2 ON ...`	`JOIN ... ON`
Unión	$\cup$	`... UNION ...`	`UNION`
Intersección	$\cap$	`... INTERSECT ...`	`INTERSECT`
Diferencia	$-$	`... EXCEPT ...`	`EXCEPT`
Renombramiento	ρ	`R AS alias`	`AS`

Ejercicios propuestos

Usando la base `peliculas.db`:

1. Nombres de actores mayores de 40 años.
2. Películas de categoría 'Drama' con calificación ≥ 8.
3. Nombres de actores que actuaron en 'Interstellar'.
4. Categorías en las que actuó 'Leonardo DiCaprio'.
5. Actores que actuaron en alguna película de 'Drama' pero en ninguna de 'Ciencia Ficción'.

Para cada uno: escribir la expresión en álgebra relacional y luego la consulta SQL.

Cómo usar el material

```
# Crear la base de datos
python3 crear_db.py

# Ejecutar las consultas interactivamente
sqlite3 peliculas.db
```